

The Traveling Tournament Problem: Valid Solutions are Different across Instance Sizes

Bas Luyen¹[0009-0002-7935-4065] Duncan Bart²[0009-0006-8096-5890]
Florian Richoux³[0000-0003-4693-379X] Daan van den Berg¹[0000-0001-5060-3342]

¹VU University Amsterdam, The Netherlands

²University of Twente, The Netherlands

³MLRT, AIRC, NAIST, Tokyo, Japan

daan@yamasan.nl

Abstract. We generate all 160 valid solutions for the traveling tournament problem with 4 teams, and randomly select 10,000 valid solutions for 6, 8 and 10 teams. For every number of teams, the difference between any two valid solutions is taken and charted. It turns out these differences are very large, even when a key constraint, the home/away assignment, is completely ignored. These results might signal that the use of meta-heuristic algorithms for this problem might be highly problematic.

1 The Traveling Tournament Problem

The Traveling Tournament Problem (TTP) is an NP-hard optimization problem in which a tournament should be scheduled for an even number of teams ($n_{teams} \geq 4$), which are dispersed across a geographical area, typically the United States [12]. The objective is to minimize the teams' travel time between venues, and the tournament should abide by three constraints (See Fig. 1):

1. Every team plays every other team twice in the tournament (once at home, once away), which is known as the **doubleRoundRobin** constraint.
2. When team A plays team B in one round, the inverse match (B playing A) cannot take place in the consecutive round, which is known as the **noRepeat** constraint.
3. The maximum number of consecutive games a team can play at home or away, is the **maxStreak** constraint. Usually, $maxStreak = 3$ meaning any team can play at most three consecutive rounds at home, or away, anywhere in the tournament.

These three constraints are so prohibitive, that a number of vexing complications arise. Just finding a valid solution is already so hard, that the usual approach of (meta)heuristic optimization for NP-hard problems appears unattainable. Uniform random sampling of valid solutions appears not to be possible in subexponential time and, possibly related, there are no known efficient mutation or crossover operators that connect the entire search space of valid individuals



Fig. 1. Left: A valid 4-team TTP tournament. **Right:** An invalid TTP tournament, having a noRepeat violation in column 2, a maxStreak=3 violation in column 4, and doubleRoundRobin violations in rounds (rows) 2, 3 and 4. The tournaments have 24 slots, of which 20 are mutable, because the first row is fixed. The difference between these two schedules is 12 out of 20 mutable slots, or 60%.

[34–36, 40]. In this study, we will suggest that the unavailability of these operators might be due to the difference between valid schedules. This has already been demonstrated for $n_{teams} = 4$, the smallest possible instance size [24], but here we will show that larger instance sizes $n_{teams} = 6, 8, 10$ also suffer these complications. We also provide some new numerical data, such as the number of valid solutions for $n_{teams} = 6$.

2 Background: algorithms for the TTP

The traveling tournament problem is a hard optimization problem, but its classification as NP-hard, meaning it has no polynomial time algorithm¹ to find the minimum travel time, was only first done in 2009 [5]. This preprint by Bhattacharyya’s pertained to the TTP without the *maxStreak* constraint, but the full proof of NP-completeness (including the *maxStreak* = 3 constraint as originally formulated by Easton & Trick) appeared in 2011 [31]. In 2021, Diptendu

¹ As long as a proof for $P = NP$ is not delivered

Chatterjee expanded the proof showing that for every $maxStreak > 3$, the problem is NP-hard [8]. The cases for $maxStreak < 3$ are weird though; even though for $maxStreak = 1$, the optimal travel distance is trivially found, the accompanying solution might not. For $maxStreak = 2$, the situation is as yet unknown. Lastly, at the time of writing this paper, it seems like a proof for APX-hardness of the TTP has been found [45]. These authors JingYang Zhao and MingYu Xiao from Chengdu University, have apparently also come up with a new approximation algorithm recently [44]. Both these findings seem relevant for our work, and need to be studied in the near future.

In much of the history of computational problem solving, programming evolutionary algorithms was the way to tackle NP-hard problems, but for the TTP, this is hardly a feasible approach. An immediate obstacle is the fact that there is no known algorithm to create a uniform random initial population in feasible time [36, 35, 34, 17]. In this study, we will show that it is also unlikely to have a mutation operation that relates valid solutions in polynomial time.

Still, a few hill climbing and simulated annealing approaches have been applied to the TTP; these could both be regarded as evolutionary algorithms with a population size of 1 [1, 23, 21] (the last one applied to the *mirrored* traveling tournament problem, a special, possibly easier variant of TTP). Also, there is a non-evolutionary approach with ant colony optimization [33], tabu search [15] and some integer programming in an experiment done by the problem’s founders themselves [13]. It should be noted that the problem of developing good mutations is very common in these studies, and as an alternative, constraints are often relaxed, or invalid (“infeasible”) solutions allowed during the search. The latter, also used by Xiang et al [40], could be a viable approach, although nobody ever reports how long their algorithms remain in invalid-space. We think it could well be forever.

A rather complex bound-driven version of beam search was developed by Nikolaus Frohner and his colleagues, augmented with a hint of randomness [14]. Nonetheless, optimal solutions found in these experimental setups usually range from $n_{teams} = 10$ to $n_{teams} = 16$, a staggeringly small number, especially when compared to the Euclidean traveling salesman problem, which can be solved exactly by a (non-general) approach up to many thousands of cities [3].

There are a few papers reporting application of a genetic algorithm to the TTP. Most of these are either non-scientific papers, or provide so little explanation on their methods that one could better start anew. A study by Baijoli et al. has mutations that possibly do not connect the entire search space, and can produce invalid individuals [6]. This problem also occurs in a study by Ribeiro and Urruaita, who sometimes have to build up home/away designations from scratch [26]. There are approaches with an artificial bee colony algorithm [28], a grey wolf optimizer [16]. These authors prevent an invalid schedule from being created by a mutation by not executing it, but they do not report how often this happens. We think it could be close to 100% of the time.

Khelifa et al. propose an “enhanced genetic algorithm for the TTP” [22], which uses the non-uniform-random polygon method for initialization [9, 11],

and mutations from Baijoli et al. [6] and introduce a new crossover. They rightfully note that “it is not easy to create the remaining rounds of an incomplete schedule without breaking the [doubleRoundRobin] constraint”, but the treatment of invalid schedules is not described further. Also, the mutation types as described on page 2 of their paper might introduce violations, but we were as yet unable to reach the authors for comments or source code. Another example of the problematic behaviour of TTP when it comes to initial populations, mutations and crossovers for metaheuristic algorithms.

Despite the plethora of metaheuristic approaches to the TTP, there are quite a number of approximation algorithms [30, 32, 39, 25, 42, 20, 41, 43, 46]. This is quite remarkable, as approximation algorithms are usually quite a bit more complicated, both conceptually and implementationally, than (meta)heuristic algorithms. Is that just a quirk, or do heavily constrained NP-hard problems like TTP in some way facilitate the development of a certain class of algorithms?

3 Finding valid schedules

As it turned out, both the search space size and the solution space size increase so violently in n_{teams} , that different methods were needed for different instance size. Assuming that each round holds each team only once, the search space contains a superexponential $(n_{teams}!)^{2(n_{teams}-1)}$ schedules which can be tested for validity. As a symmetry breaking measure however, all tournaments were created ‘half-normalized’, meaning that the first round was always fixed to hold the matchups $(t_0, t_1), (t_2, t_3), \dots, (t_{n_{teams}-2}, t_{n_{teams}-1})$. This causes the search space to shrink by a factor $\frac{(n_{teams}!)!}{(1/2n_{teams})!}$, which represents the number of possible valid rounds in a tournament. It should be noted however, that this measure is only acceptable because this study is concerned with finding *valid* rather than *optimal* schedules. In other words: as soon as travel distances are involved, this method of symmetry breaking is no longer possible as the actual permutation of the first round matters for the travel distance.

After fixing the first round for symmetry breaking, the algorithm recursively tries all $n_{teams} \cdot (n_{teams} - 1)$ combinations of matchups for each round, but after the placement of any single matchup in both relevant columns however, deploys forward checking to decide whether any of the 3 constraints in the partial schedule are already violated. If so, the search on that subbranch is halted and the algorithm backtracks, at most until just before the first round, which is fixed (See pseudocode in Algorithm 1).

More particular, *maxStreak* violations are prevented during search before they occur. This is done in two ways: first, by not placing another away game² that would immediately break the maxStreak constraint, but second, by keeping track of the number of home and away games that still need to be placed in a column during the buildup of the tournament. For any column, *maxLeft* denotes the number of either the homes games left to place, or the away games left to

² or home game

place, whichever is largest. For the same column, $minLeft$ denotes the smaller alternative. Then if $\frac{maxLeft}{3} > minLeft + 1$, there are too many home or away games left to play and a $maxStreak$ violation *must* occur at some point in the future of the search. So if a team still has 1 home game and 8 away games left, there is no way to arrange these last 9 games in such a way that a $maxStreak$ violation will not occur for $maxStreak = 3$. If a partial schedule fails on any of these two $maxStreak$ checks, the algorithm halts and backtracks.

Algorithm 1 The procedure **WorkerFunc** is the function performed in parallel on each node. It recursively tries adding matchups to the current permutation of the schedule. *StreakCount* keeps track of the home and away games played in a row by each team and how many home and away games each team still has left to play in the current schedule.

```

1: Global: count
2: procedure WORKERFUNC(schedule, matchups, streakCount)
3:   if schedule is complete then
4:     count++
5:     return
6:   end if
7:
8:   for all m in matchups do
9:     if m CHECKCONSTRAINTS(schedule, m, matchups, streakCount) then
10:      continue
11:    end if
12:
13:    schedule.append(m)
14:    matchups.remove(m)
15:    UPDATESTREAK(streakCount, m)           ▷ Update home/away streak
16:
17:    WORKERFUNC(schedule, matchups, streakCount)   ▷ Recursive call
18:
19:    schedule.remove(m)
20:    matchups.insert(m)
21:    REVERTSTREAK(streakCount)                 ▷ Revert to previous state
22:  end for
23: end procedure

```

Prevention of *noRepeat* violations, happens in two similar ways: first, a inverted matchup (t_a, t_b) is not placed if the previously constructed round hold the matchup (t_b, t_a). Second, when starting the penultimate round, a check through all remaining unplaced matchups is performed. If for any matchup (t_i, t_j) the inverse matchup (t_j, t_i) also still needs to be placed, a *noRepeat* violation is unavoidable, and the algorithm halts and backtracks.

Although depth-first search is memory efficient and relatively fast (especially after all the applied pruning techniques) it is by nature still an (su-

per)exponential time algorithm, and the Traveling Tournament Problem makes that very tangible for all instance sizes of $n_{teams} > 4$. For $n_{teams} = 6$, the search space contains $\approx 3.74 \cdot 10^{28}$ schedules, a number far too large to search on a single computer, even with good pruning techniques. We managed to find all valid schedules though, by deploying an initial breath-first search (BFS) to exhaustively produce all valid *partial schedules with 5 matchups filled out*. Upon generation, each partial schedule with 5 valid rounds is wrapped into a ‘task’ and then distributed among 16 nodes with 16 cores on the Snellius cluster for further completion with our previously described depth-first routine. Once a node finishes a task, either by completing a schedule or asserting its incompleteness, it gets a new task from the queue, unless the queue is empty. Emptying the queue took less than half a minute for $n_{teams} = 6$, and we saved all 54,427,728 valid schedules to disk, in a file growing to over 6GB in size. From this file, 10,000 unique valid half-normalized schedules were randomly selected for pairwise comparison. Although it might technically be possible to do an exhaustive pairwise comparison for all 1,481,188,760,407,130 schedule pairs, we deemed it infeasible within the scope of this investigation. Considering the comparison for 10,000 schedules already took several hours, a full mapping of all schedule pairs would take 3381 years – in a conservative estimate.

To further enhance runtimes and memory, the tournament’s data structure is not deepcopied between recursive calls. Instead, pointers are passed between recursions, and a list of undo operations is executed after each recursive call. The core of the algorithm has been written in C++, even though some comparison scripts are currently still in Python. All source code can be found in our public repository [2]. Still, for $8 \geq n_{teams} \geq 10$, these measures did not suffice to count all valid half-normalized schedules in any kind of feasible time.

For $n_{teams} = 8$, we initially tried to perform an exhaustive search through its search space of $\approx 3.00 \cdot 10^{64}$ schedules, but failed hopelessly. We first ran the initial breadth-first search with a depth of 5, producing 42,240 tasks, of which fewer than 1300 were finished after 5 days of runtime on 6 nodes with 128 cores, and yielding not a single valid schedule. As such, we abandoned this approach and opted for “random backtracking”, which entails a normal depth-first search, but with matchups placed in a prefixed random order, different for each round. The term ‘prefixed’ means that the random order of matchups placement for each round was fixed throughout one run. In other words, the search was still exhaustive, but traversed the search space not necessarily from $(t_0, t_1), (t_0, t_2), \dots, (t_{n_{teams}-2}, t_{n_{teams}-1})$, but for instance, as $(t_4, t_2), (t_1, t_7), \dots, (t_0, t_3)$ for a round. However, when the algorithm backtracked, it was still the next matchup from this prefixed randomized order that was tried, and *not* re-randomized during the DFS run. After a first valid schedule was found, the search was halted, and started anew for a second schedule, with new random matchup orders. In total, this process was repeated 10,000 times, giving us a random selection of 10,000 valid schedules in approximately 2.5 hours on a single core.

For $n_{teams} = 10$, we never tried an exhaustive search. Like for $n_{teams} = 8$, random backtracking was deployed to obtain 10,000 randomly selected valid

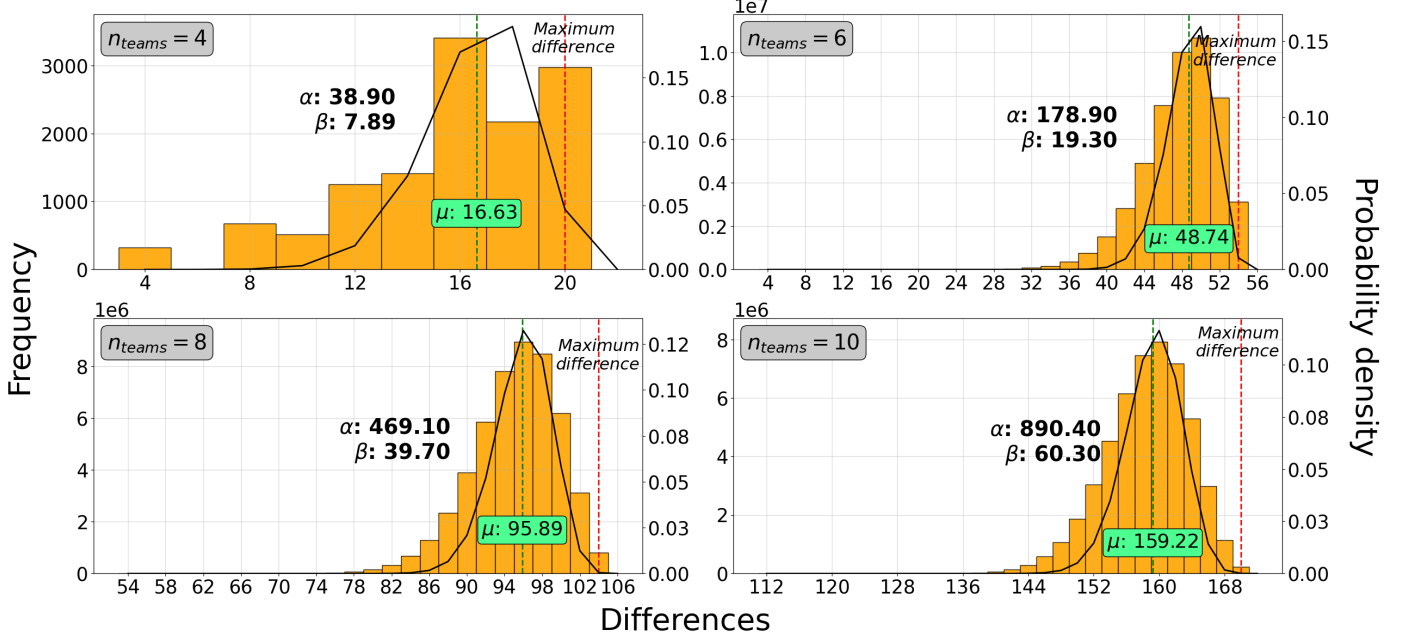


Fig. 2. Valid schedules for the Traveling Tournament Problem of instance sizes $n_{teams} = 4, 6, 8$ and 10 are almost maximally different. This means that in two randomly chosen schedules, almost every slot either holds a different team, a different home/away designation, or both. For this reason, optimization algorithms are highly unlikely to mutate one valid solution into another.

schedules from its search space of $\approx 1.19 \cdot 10^{118}$ schedules. This time, the process took approximately 40 hours on a single core, a surprisingly small amount of time.

4 Three ways of comparing valid schedules

The difference between two valid half-normalized schedules is calculated on a per-slot basis, which means if a slot on location $[Column, Row]$ holds either a different team, a different home/away designation or both, the slot is counted as 'different'. The difference between two schedules is the total number of different slots, where the first round is ignored (since it is fixed and equal from half-normalization). As an example, both schedules from Figure 1 have 24 slots, of which 20 are mutable. The maximum difference between these schedules is therefore 20. With 12 slots unequal the difference between these two is 60%.

Second, we also compared every two schedules with their home/away designations *removed*. The idea behind it is to find out whether it makes sense to first find a valid schedule just abiding by the doubleRoundRobin-constraint and

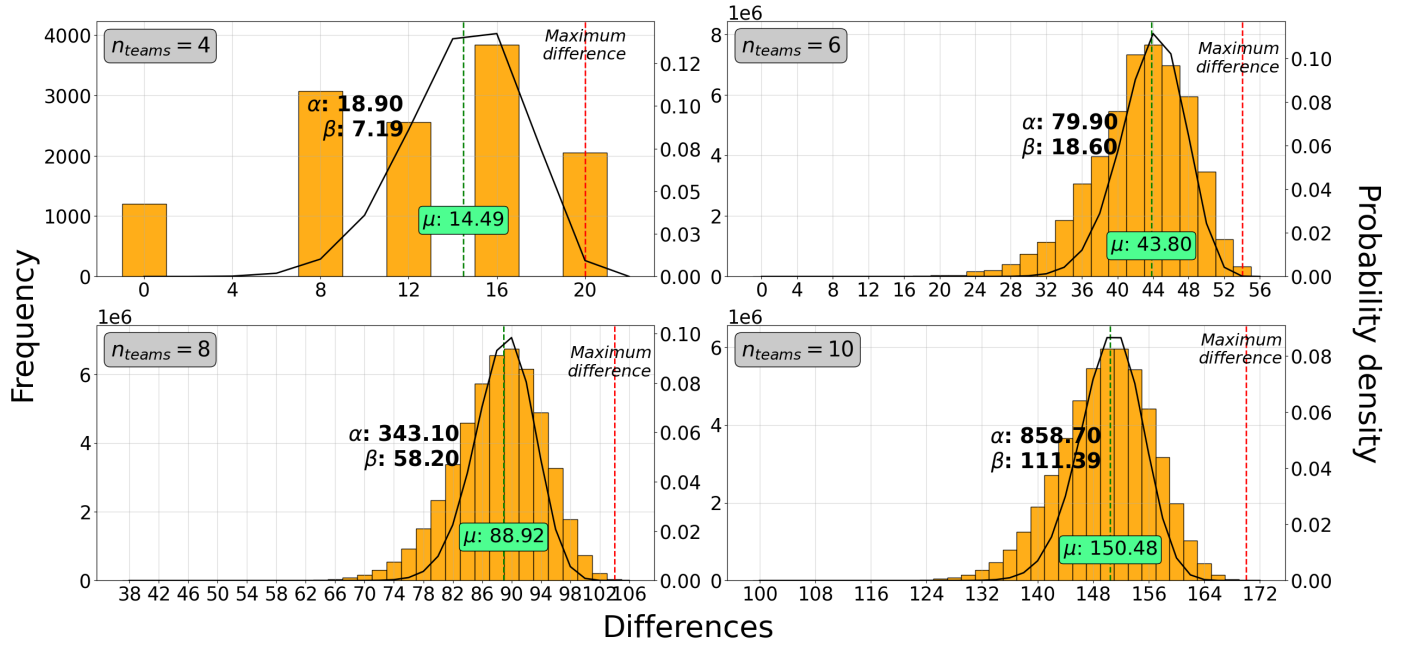


Fig. 3. Differences between valid tournaments for $n_{teams} = 4, 6, 8$ and 10 *without* home/away assignments are slightly less different on average, but still close to maximal. This suggests that finding a valid doubleRoundRobin-schedule without repeats first, and the affixing the home/away assignments afterwards does not make the problem substantially easier.

the noRepeat constraint, and find a valid home-away plan afterwards. Spoiler: it isn't.

Third, we also did the inverse: removing the team designations and leaving *only* the home/away assignments for calculation of the differences. Note that the second and third measure of difference are not complementaries of the first, and can therefore not be added up. Rather, they are disjunctive: a slot as a whole is different (S) if either the allocated team is different (T), or the home/away assignment is different (HA), or both. Therefore, it follows that $|S| \leq |T| + |HA|$. The question associate with this principle is whether it makes sense to first construct a home/away skeleton, and put the team assignments in later. Again, sadly, the answer appears to be no, as we will shortly see in Section 5.

The histogram of all three difference types showed somewhat convergent behaviour, encouraging fitting of a beta-binomial curve through these distributions, allowing for skewness of the distribution. The exception is the third comparison (the home/away only), which was fit with a normal curve because of its apparent symmetry.

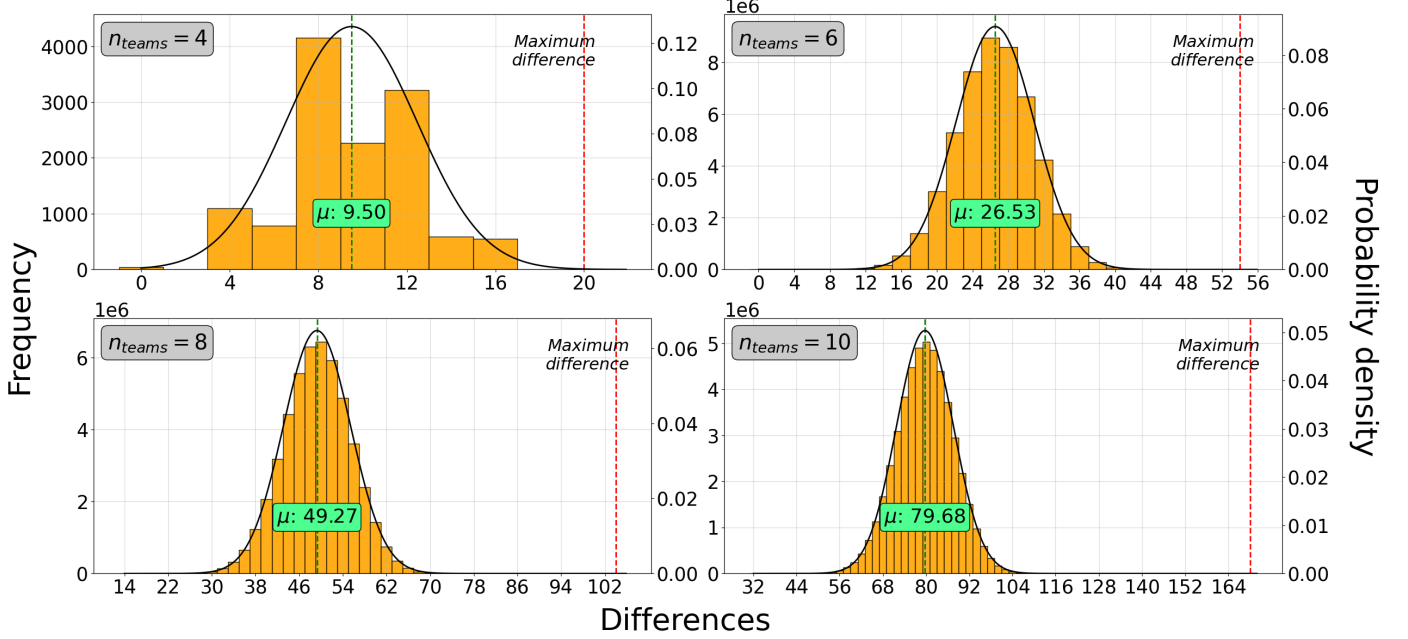


Fig. 4. Differences between valid tournaments for $n_{teams} = 4, 6, 8$ and 10 *only* considering the home/away assignments are about half the maximum difference.

5 Results

5.1 Overall difference in schedules

For $n_{teams} = 4$, the search space contains ≈ 191 million half-normalized schedules with 0 or more violations, which can all be checked in a few seconds. Of these, 160 schedules are violation free and therefore valid. This number is small enough to exhaustively make a full comparison of all $\frac{160 \times 159}{2} = 12,720$ schedule pairs. The two schedules in a pair can have at most 20 of their 24 slots different, as the first round is fixed. A remarkable 2926 ($\approx 23.3\%$) of these 12,720 schedule pairs actually *have* that maximal difference of 20 slots. The other schedule pairs are also very different with an average of $\mu = 16.83$ different slots. This means that two valid schedules for $n_{teams} = 4$, on average, are over 84% different! Only 320 pairs have the lowest found difference of 4 slots, a meagre $\approx 2.3\%$.

For higher instance sizes the numbers get even worse. For $n_{teams} = 6$, the 49,995,000 schedule pairs from the 10,000 schedules randomly sampled from the $\approx 54M$ valid half-normalized schedules, the highest number of different 54 slots was found 3,108,103 times, which accounts for $\approx 6.2\%$. The smallest difference found (4 different slots) was found only 10 times ($\ll 0.1\%$), but the most frightening number is the average difference of $\mu = 48.74$ slots, implying that two valid schedules of 6 teams are, on average, 90.3% different.

The trend continued further upward for larger instance sizes, where two valid schedules for $n_{teams} = 8$ are 92.2% different, and for $n_{teams} = 10$ are 93.7% different on average. Percentages for ‘minimum difference pairs’ and ‘maximum difference pairs’ dropped for these instance numbers though, signaling a contracting distribution (Fig. 2, bottom row) with an increasing average.

5.2 Differences in schedules without home/away designations

Removing the home/away designations from the comparison hardly improved the matter. For $n_{teams} = 4$, the average difference for two schedules dropped to $\mu = 14.49$ from $\mu = 16.83$ which still signals an average of 72.5% (down from 84.2%). For $n_{teams} = 6$, these numbers amounted to $\mu = 43.8$ from $\mu = 48.7$ which still signals an average of 81.1% (down from 90.3%). For $n_{teams} = 8$, the difference without home/away-assignments is 85.5% (down from 92.2%) and for $n_{teams} = 10$ it’s 88.5% from 93.7% (Figure 3).

The interesting pattern here is that the difference percentages from Figures 2 and 3 become closer as the instance size n_{teams} increases. If this trend is universal, it might mean that asymptotically speaking, the Traveling Tournament Problem without home/away designations might be just as hard as with the home/away constraint, at least when it comes to finding valid solutions to begin with. For the problem as a whole however, this question could be moot, as removing home/away designations (e.g. by assuming all teams meet in some central location every round of the tournament) also removes differences in travel time, relegating the constraint optimization problem to an easier constraint satisfaction problem of just finding a valid schedule. The word ‘easier’ however, should not be taken too literally in this sense.

5.3 Difference in home/away designations alone

When we remove the *teams* from the valid schedules and leave only the home/away designations, things look a bit different. The average difference is remarkably enough still relatively high: $\mu = 9.50$ for $n_{teams} = 4$, and $\mu = 26.53, 49.27, 79.68$ for $n_{teams} = 6, 8, 10$, which corresponds to differences of 47.5%, 49.1%, 47.4% and 46.9% respectively. This means that on average, two valid schedules have just about as many homes/away assignments in common as would be produced by a complete random guess (Figure 4).

But different from the other distributions of differences, is that this one is apparently *symmetric*, as can be seen in Figure 4. The respective standard deviations $\sigma = 2.97, 4.40, 6.17, 7.92$ increase, but not when related to the range of the maximum number of home/away differences. In other words: relative to their range, the distributions get narrower.

6 Implications

The enormous difference between valid solutions of the TTP optimization problem poses a major difficulty. Being NP-hard, it is not realistically solvable with

exact algorithms, but our findings suggest it might also be not solvable with evolutionary algorithms. The reason for this, we conjecture, is the unavailability of a realistic mutation operator. Applying a change that modifies more than 80% of a valid solution is so close to random sampling, that a hill climber is very unlikely to be successful as most mutations would be rejected. Second, such large changes are also likely to have an enormous impact on the travel time. Third, whether such an operator can be described in a reasonably short procedure (a function in C++ or the like) is also very questionable. After all, mutations are *general* procedures, such as swapping two cities in a traveling salesman problem, two tasks in job shop scheduling, or any low number of integers in the partition or subset sum problem [19, 18, 29, 27, 4]. Those mutations work on *every* valid solution of *every* instance subjected to it, and 100% surely turn it into another valid solution in deterministic polynomial time. Furthermore, these mutations provably connect the entire solution space. Our results suggest that for the TTP, such a general procedure might not be available at all. This might be the reason that most, if not all mutations in literature for the TTP are either non-general, have a mandatory repair function, or both. Besides, whether they 100% surely turn a valid solution into another valid solution in polynomial time is highly debatable. Lastly, it is very unlikely these existing mutation operators connect the entire solution space.

To make things worse, there is also no known random sampling method for the traveling tournament problem. One reason emerging from our results could be the extreme sparsity of valid solutions for an instance. For just 6 teams, we exhaustively counted 54,427,728 zero-conflict schedules, which is not even a spec in the universe of its $3.27 \cdot 10^{24}$ item wide search space. Initially, we also hoped to count the number of valid schedules for $n_{teams} = 8$ and 10 but sadly, we couldn't get it done even for these small numbers. They might be even more sparse, but as yet, we cannot say for sure.

Concludingly, it looks like the TTP is not only unsolvable for exact algorithms, but for metaheuristic algorithms too. Its solutions are just too few and far between, but mostly too different. And the latter property might just mean there are no structural ways of either sampling or mutating its solutions, both of which are critically needed for a metaheuristic approach.

7 Future Work

Along the paper decision, ECTA's reviewers have supplied a remarkable number of ideas, which we list as intended future work, either for us or for other teams.

- An investigation could be launched into how these findings might directly inform the design or tuning of actual solvers. We are sceptical, but nonetheless some evolutionary algorithms, like the Plant Propagation Algorithm that explores-exploits both locally and globally without crossover, might be suited – and therefore tunable to this problem [10, 7, 37, 38]. The problem of finding search space covering mutations still remains though. But if these mutations exist, even if just stochastically, this avenue appears worth exploring.

- One reviewer suggested to replicate existing approaches. After some initial reservations, we think there could be merit in this approach, especially when such replications are approached more data-heavily than the originals. How often do invalid solutions appear? For how many iterations on end? How often do mutation types fail? These measurements could typically bridge the findings in our study with previously reported results.
- A characterization and extrapolation to larger instance sizes should be conducted. This is indeed a promising way forward, and could be rather easy as the algorithms are already written. An obstacle to the approach however, is the required computational effort. However, of all suggestions, this might be the most feasible next step.
- One reviewer suggested statistical thresholds for interpretation of the effects witnessed. This is indeed a good idea, and should follow the previous point because at this time, we don't know what to expect in terms of results when instance sizes increase.
- One reviewer suggested an investigation into 'repeating motives' and 'recurring patterns'. Indeed it is interesting to conduct a more extensive comparison than slot-by-slot as in the current investigation, and the approach could also shed some light on feasibility of mutation operators. One way forward could be given by approaching the edit distance between every two schedules for any n . Again, computationally heavy, but feasible if one settles for approximate answers rather than exact results, and a feasible way forward.

We thank ECTA's reviewers for these suggestions, and will undoubtedly explore these avenues in the future. Please feel free contact us if any further suggestions arise.

8 Credit statement and disclaimer

Bas Loyen: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data Curation, Writing - Original Draft, Writing - Review & Editing, Visualization. **Duncan Bart:** Methodology, Software, Validation, Writing - Review & Editing. **Florian Richoux:** Writing - Review & Editing. **Daan van den Berg:** Conceptualization, Resources, Writing - Review & Editing, Supervision, Project administration.

The right-hand side of Figure 1 serves an illustrative purpose; we suspect that any sensible row-first algorithm cannot create invalid tournaments with 1,2 or 3 noRepeat violations for $n_{teams} = 4$. Python and C++ source code, and auxiliaries to this publication are publicly available [2]. The logos in this figure were (re)made by melling2293@Flickr and are distributed under Creative Commons license. An initial segment of this paper was published as a late-breaking abstract at Evostar 2025 in Italy [24].

References

1. Anagnostopoulos, A., Michel, L., Hentenryck, P.V., Vergados, Y.: A simulated annealing approach to the traveling tournament problem. *Journal of Scheduling* **9**, 177–193 (2006)
2. Anonymous: Repository containing source material: <https://github.com/Temp12345-del/TTP-DFS> (2025)
3. Applegate, D.L., Bixby, R.E., Chvátal, V., Cook, W., Espinoza, D.G., Goycoolea, M., Helsgaun, K.: Certification of an optimal tsp tour through 85,900 cities. *Operations Research Letters* **37**(1), 11–15 (2009)
4. Van den Berg, D., Adriaans, P.: Subset sum and the distribution of information. In: *IJCCI*. pp. 134–140 (2021)
5. Bhattacharyya, R.: A note on complexity of traveling tournament problem. *Optimization Online* **2480** (2009)
6. Biajoli, F.L., Lorena, L.A.N.: Mirrored traveling tournament problem: An evolutionary approach. In: Sichman, J.S., Coelho, H., Rezende, S.O. (eds.) *Advances in Artificial Intelligence - IBERAMIA-SBIA 2006*. pp. 208–217. Springer Berlin Heidelberg, Berlin, Heidelberg (2006)
7. Brouwer, N., Dijkzeul, D., Koppenhol, L., Pijning, I., Van den Berg, D.: Survivor selection in a crossoverless evolutionary algorithm. In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. pp. 1631–1639 (2022)
8. Chatterjee, D.: Complexity of traveling tournament problem with trip length more than three. *arXiv preprint arXiv:2110.02300* (2021)
9. de Werra, D.: Some models of graphs for scheduling sports competitions. *Discrete Applied Mathematics* **21**(1), 47–65 (1988). [https://doi.org/https://doi.org/10.1016/0166-218X\(88\)90033-9](https://doi.org/https://doi.org/10.1016/0166-218X(88)90033-9), <https://www.sciencedirect.com/science/article/pii/0166218X88900339>
10. Dijkzeul, D., Brouwer, N., Pijning, I., Koppenhol, L., Van den Berg, D.: Painting with evolutionary algorithms. In: *International Conference on Computational Intelligence in Music, Sound, Art and Design (Part of EvoStar)*. pp. 52–67. Springer (2022)
11. Dinitz, J.H., Froncek, D., Lamken, E.R., Wallis, W.D.: Scheduling a tournament. In: *Handbook of Combinatorial Designs*. Chapman and Hall/CRC (Nov 2006). <https://doi.org/10.1201/9781420010541>, <http://dx.doi.org/10.1201/9781420010541>
12. Easton, K., Nemhauser, G., Trick, M.: The traveling tournament problem description and benchmarks. In: *Principles and Practice of Constraint Programming—CP 2001: 7th International Conference, CP 2001 Paphos, Cyprus, November 26–December 1, 2001 Proceedings 7*. pp. 580–584. Springer (2001)
13. Easton, K., Nemhauser, G., Trick, M.: Solving the travelling tournament problem: A combined integer programming and constraint programming approach. *Lecture notes in computer science* pp. 100–112 (2003)
14. Frohner, N., Neumann, B., Raidl, G.: A beam search approach to the traveling tournament problem. In: *Evolutionary Computation in Combinatorial Optimization*. pp. 67–82. Springer International Publishing (2020)
15. Gaspero, L.D., Schaerf, A.: A composite-neighborhood tabu search approach to the traveling tournament problem. *Journal of Heuristics* **13**, 189–207 (2007)
16. Gupta, D., Anand, C., Dewan, T.: Enhanced heuristic approach for traveling tournament problem based on grey wolf optimizer. In: *2015 Eighth International Conference on Contemporary Computing (IC3)*. pp. 235–240 (2015). <https://doi.org/10.1109/IC3.2015.7346685>

17. Hassel, L., Verduin, K., Loyen, B., Thomson, S.L., Weise, T., van den Berg, D.: The traveling tournament problem: Constraint violations for different maxstreak values (2025)
18. Horn, R., Jansen, R., van Eck, O., van Den Berg, D.: Separating the yes-from the no-instances in the number partitioning problem. In: 16th International Joint Conference on Computational Intelligence, IJCCI 2024. pp. 181–188. SciTePress (2024)
19. Horn, R., Thomson, S.L., Van Den Berg, D., Adriaans, P.: The easiest hard problem: Now even easier. In: Proceedings of the Genetic and Evolutionary Computation Conference Companion. pp. 97–98 (2024)
20. Imahori, S., Matsui, T., Miyashiro, R.: A 2.75-approximation algorithm for the unconstrained traveling tournament problem. *Annals of Operations Research* **218**(1), 237–247 (2014)
21. Jha, S., Menon, V.: Bbmttp: Beat-based parallel simulated annealing algorithm on gpgpus for the mirrored traveling tournament problem. In: Proceedings of the High Performance Computing Symposium. pp. 1–7 (2014)
22. Khelifa, M., Boughaci, D., Aïmeur, E.: An enhanced genetic algorithm with a new crossover operator for the traveling tournament problem. In: 2017 4th International Conference on Control, Decision and Information Technologies (CoDIT). pp. 1072–1077 (2017). <https://doi.org/10.1109/CoDIT.2017.8102741>
23. Lim, A., Rodrigues, B., Zhang, X.: A simulated annealing and hill-climbing algorithm for the traveling tournament problem. *European Journal of Operational Research* **174**(3), 1459–1478 (2006)
24. Loyen, B., Hassel, L., Thomson, S.L., Weise, T., van den Berg, D.: The traveling tournament problem: Valid solutions are very different. *LBAs of EvoSTAR 2025* (2025)
25. Miyashiro, R., Matsui, T., Imahori, S.: An approximation algorithm for the traveling tournament problem. *Annals of Operations Research* **194**, 317–324 (2012)
26. Ribeiro, C.C., Urrutia, S.: Heuristics for the mirrored traveling tournament problem. *European Journal of Operational Research* **179**(3), 775–787 (2007)
27. Salas, J.: Beyond worst-case subset sum: An adaptive, structure-aware solver with sub- $2^{n/2}$ enumeration. *arXiv preprint arXiv:2503.20162* (2025)
28. Saul, S., Adewumi, A.: An artificial bees colony algorithm for the traveling tournament problem. In: 41st Annual Conference of the Operations Research Society of South Africa. p. 10 (2012)
29. Sazhinov, N., Horn, R., Adriaans, P., van den Berg, D.: The partition problem, and how the distribution of input bits affects the solving process. In: IJCCI. pp. 143–150 (2023)
30. Thielen, C., Westphal, S.: Approximating the traveling tournament problem with maximum tour length 2. In: Algorithms and Computation: 21st International Symposium, ISAAC 2010, Jeju, Korea, December 15–17, 2010, Proceedings, Part II 21. pp. 303–314. Springer (2010)
31. Thielen, C., Westphal, S.: Complexity of the traveling tournament problem. *Theoretical Computer Science* **412**(4–5), 345–351 (2011)
32. Thielen, C., Westphal, S.: Approximation algorithms for ttp (2). *Mathematical Methods of Operations Research* **76**(1), 1–20 (2012)
33. Uthus, D.C., Riddle, P.J., Guesgen, H.W.: An ant colony optimization approach to the traveling tournament problem. In: Proceedings of the 11th Annual conference on Genetic and evolutionary computation. pp. 81–88 (2009)

34. Verduin, K., Horn, R., van Eck, O., Jansen, R., Weise, T., van den Berg, D.: The traveling tournament problem: Rows-first versus columns-first **1**, 447–455. <https://doi.org/DOI: 10.5220/0012557700003690>
35. Verduin, K., Thomson, S.L., van den Berg, D.: Too constrained for genetic algorithms. too hard for evolutionary computing. the traveling tournament problem. In: Proceedings of the 15th International Joint Conference on Computational Intelligence ECTA - Volume 1. pp. 246–257 (2023). <https://doi.org/10.5220/0012192100003595>
36. Verduin, K., Weise, T., van den Berg, D.: Why is the traveling tournament problem not solved with genetic algorithms? (2023)
37. Vrielink, W., Van den Berg, D.: A dynamic parameter for the plant propagation algorithm. *Evo* LBA's* pp. 5–9 (2021)
38. Vrielink, W., Van den Berg, D.: Parameter control for the plant propagation algorithm. *Evo* LBA's* pp. 1–4 (2021)
39. Westphal, S., Noparlik, K.: A 5.875-approximation for the traveling tournament problem. *Annals of Operations Research* **218**, 347–360 (2014)
40. Xiang, C., Wu, Z., van den Berg, D., Weise, T.: Randomized local search vs. nsga-ii vs. frequency fitness assignment on the traveling tournament problem (2024)
41. Xiao, M., Kou, S.: An improved approximation algorithm for the traveling tournament problem with maximum trip length two. In: 41st International Symposium on Mathematical Foundations of Computer Science (MFCS 2016). Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik (2016)
42. Yamaguchi, D., Imahori, S., Miyashiro, R., Matsui, T.: An improved approximation algorithm for the traveling tournament problem. *Algorithmica* **61**(4), 1077–1091 (2011)
43. Zhao, J., Xiao, M.: The traveling tournament problem with maximum tour length two: A practical algorithm with an improved approximation bound. In: *IJCAI*. pp. 4206–4212 (2021)
44. Zhao, J., Xiao, M.: A 5-approximation algorithm for the traveling tournament problem. *Annals of Operations Research* **346**(3), 2287–2305 (2025)
45. Zhao, J., Xiao, M.: The apx-hardness of the traveling tournament problem. *Operations Research Letters* p. 107311 (2025)
46. Zhao, J., Xiao, M., Xu, C.: Improved approximation algorithms for the traveling tournament problem. In: 47th International Symposium on Mathematical Foundations of Computer Science (MFCS 2022). Schloss Dagstuhl-Leibniz-Zentrum für Informatik (2022)